

Given: Force & Torque, Sound & Vibration

Tool Wear:

- Cycle: the machine cycle, represents a single process.
- Cutting edge: The tools used in this dataset have 4 cutting edges. Number 1 in the side teeth and the end teeth represent the same cutting edge.
- VBmax: the max value of flank wear
- VB in 1/2ap: the value of flank wear in the 1/2 cutting depth
- S: the wear area, mm²

Questions:

- How does each column in the tool wear csv relate to the actual tool wear? How do we characterize or define the tool wear?
 - Important data: flank wear value & wear area
 - Which one to use? Use both? how?
 - Wear width (VBmax) for now, because it captures the worst local wear
 - Usually the standard wear indicator
 - Failure threshold or tool replacement criteria
- How to relate the given to tool wear?
 - Tweak:
 - Number of layers
 - Number of neurons in each layer?
 - learning rate

Plan:

- Process the data to:
 - output: Only the wear width (averaged for all edges) & cycle
 - input:
 - force x, y, z & torque to force (averaged) & torque
 - vibration data xyz + sound data + α ? to vibration data (averaged) & torque?
 - Since the input data is too large (millions of data points for one cycle)
 - Time-domain (8 features per channel):
 - Mean, Standard deviation, RMS, Max, Min, Peak-to-peak, Skewness, Kurtosis
 - Frequency-domain (3 features per channel):
 - Dominant frequency (from FFT), Spectral centroid, Total spectral energy
- MLP because the datasets are only 68. CNN would be overfit

Execution

- skip one data with no sound & vibration but only force

Result:

- Most predictions are within 0.02-0.03 mm of the actual value. The worst prediction is

sample 5 (actual 0.2625, predicted 0.1735, error of -0.089 mm), which is the highest-wear sample in the test set. The best prediction is sample 9, off by only 0.0014 mm

- corrupted data → might be caused when downloading
 - so I can download again and process the data again
- windowed samples containing the data from when it is not cutting as well

4/20 update

- Leah: 89.46 %
 - Output: Vb max averaged from all 8 sides
 - **Might be better to only consider the edge (since end doesn't get worn that often)**
 - Input: characteristics of all force & torque and sound & vibration
 - Characteristics
 - Mean, Standard Deviation, Max, Min, RMS, Kurtosis, Skewness, Peak to peak, Crest factor, Zero crossing rate, 25th percentile, 75th percentile, frequency domain, dominant frequency, mean spectral energy (15)
 - Mean frequency Weighted average frequency Center of mass of spectrum; shifts with wear
 -
 - Comment: Epoch might be too high (>300 might be overfitting)
 - TODO: try with different characteristics
 - go through the report
 - nonlinearity & other optimizers?

4/27/2026 Notes

- For Presentation:
 - Read through the code and actually learn what was going on
 - prepare for questions (read what I wrote in the document)
 - interpret the result curve (why the errors are like that)
- one more table comparing with other paper
- input only from force and torque
- fill out the table for the result
-

4/30/2026 Presentation Preparation

- todo:
 - ask claude to make report for everything in the code, how it works, and why we decided that. I need to understand what is going on more thoroughly. explain everything in basic neural network perspective.
 - read the report I put in the shared doc
 - read notes I have above
 - compare the two plots of before and after the changes

- layers and how they work
- put the backup slides
 - exact neural network architecture, the training and testing exact values from the terminal
 - put the trained values from the 200th epoch in the slide
- time for training put it into the slides
- screen shot again and put it in the backup slides
- Plot interpretation: Only force & Torque
 - Loss curves
 - Both train and test MSE drop fast in the first 30 epochs and then plateau
 - The model converges without obvious overfitting since the two curves stay close together. No divergence between them means the network is not memorizing the training set
 - Predicted vs. actual
 - The model successfully predicts the upward going behavior → meaningful wear information from force and torque
 - R^2 is moderate: half of the variance in tool wear is explained. Since vibration and sound encode high frequency chatter and surface finish degradation that that force data can't capture
 - cluster around actual V_{bmax} around 0.185, and model consistently overpredicts. Maybe because these are the cycles where the tool is in a mid life wear regime and force signals have not changed much yet.
 - At the high wear end, the model does pick up the trend but still undershoots significantly. likely because the force signals do shift but they saturate.
 - The low-wear region (0.11-0.13) is also systematically overpredicted, suggesting the baseline force features have a floor the model can't push below
 - Some windows from the same cycle predict better than others because the force torque signals are not uniform across the cut. It tends to be more uniform across the cut
 - Overall, the model clearly learns a monotonic relationship with wear
- Compare the plots
 - with sound and vibration: starting MSE is much higher
- Training / Testing process
 - 200 epochs training
 - each epoch, the model does a forward and backward pass on the training set and evaluates on the held out test set
 - After training, it takes the final training model and does one last prediction on every sample in the test set
- General knowledge:
 - There are 280 samples: 14 cycles in the test set * 20 time windows
 - R^2 is coefficient of determination, and it measures how much of the variance in the actual values the model's predictions explain

- $R^2 = 1 - \frac{\Sigma(y_{actual} - y_{predicted})^2}{\Sigma(y_{actual} - y_{actual, average})^2}$
- $R^2 = 1 \rightarrow$ every prediction is exactly right (no error)
- $R^2 = 0 \rightarrow$ model is no better than just randomly guessing
- $R^2 = 0.525$: captures about 52.5% of the variance in wear, half of the spread in the data is accounted for by the model
- Forward and backward pass
 - forward pass is feed the input through the network and compute the output and the loss.
 - backward pass is compute the gradient of the loss with respect to every weight in the network (via back propagation), then update those weights using the optimizer (adam)
 - one epoch = doing forward and backward pass for one batch
- Overfitting:
 - it means that the model memorizes the training data including noise instead of learning the general pattern. In the loss plot, the train curve keeps going down, but the test curve starts going back up at some point and the gap between them widens.
- machine learning basics:
 - machine learning: any algorithm that learns patterns from data instead of being explicitly programmed
 - Neural network: one type of machine learning. inspired by biological neurons, made of layers of simple computational units connected together
 - MLP is the simplest type of neural network. Perceptron is a old name for a single artificial neuron.
- we have 3 hidden layers + 1 output layer
- This is a supervised regression model,
 - supervised: model is trained with labeled data (known answer). So model learns by comparing those predictions to the known answers and adjusting.
 - Regression: output is a continuous number. instead of predicting worn/not worn or yes/no (which is a classification), the model output a worn value (number)
 - Specific architecture is an MLP (simplest type of feedforward neural network)
- We didn't use CNN because we have a relatively small set of data, 68 cycles, 20 time frames, and 10 for each features. For raw data, we could have used CNN.

- Claude:

Tool Wear Prediction Neural Network

Data Overview

- **Target (output):** `data/tool_wear.xls` -- has columns: Cycle, Cutting edge 1-4, Vbmax (mm), etc. We average Vbmax across edges per cycle.
- **Input source 1:** `data/Force and torque data/` -- 68 `.txt` files (tab-delimited), columns: Time, Fx, Fy, Fz, Mz. ~10 kHz sampling, 4-6M rows per file.
- **Input source 2:** `data/Vibration and sound data/` -- 59 `_processed.csv` files, columns: Time, AI1-01 (m/s²), AI1-02 (m/s²), AI1-03 (m/s²), AI1-07 (Pa). ~10 kHz sampling, 500-5M rows per file.
- Each file in both folders corresponds to **one machining cycle**.

Key Challenge: File-to-Cycle Mapping

There are 68 force/torque files and 59 vibration/sound files -- they don't match 1:1. The plan assumes:

- Files sorted chronologically by date-run in the filename (e.g., `01-26-1` before `01-27-2`) correspond to Cycle 1, 2, 3, ... in tool wear.xls.
- Only cycles that have **both** a force/torque file and a vibration/sound file will be used for training (the intersection).
- A verification step will print the mapping for manual review.

Key Challenge: Processing Raw Time Series

Each file has millions of data points -- far too large to feed directly into a neural network. The standard approach is to **extract statistical features** that summarize each file into a fixed-length vector.

Features to extract per signal channel:

Time-domain (8 features per channel):

- Mean, Standard deviation, RMS, Max, Min, Peak-to-peak, Skewness, Kurtosis

Frequency-domain (3 features per channel):

- Dominant frequency (from FFT), Spectral centroid, Total spectral energy

This gives **11 features x 4 force/torque channels + 11 features x 4 vibration/sound channels = 88 input features per cycle**.

Architecture

None

```

    o flowchart LR
    o     subgraph inputProcessing [Input Processing]
    o         FT["Force/Torque TXT\n(Fx, Fy, Fz, Mz)"]
    o         VS["Vibration/Sound CSV\n(accel x3, sound)"]
    o     end
    o
    o     subgraph featureExtraction [Feature Extraction]
    o         FTFeat["44 features\n(11 per channel x 4)"]
    o         VSFeat["44 features\n(11 per channel x 4)"]
    o     end
    o
    o     subgraph neuralNet [Neural Network - MLP]
    o         Input88["Input Layer (88)"]
    o         H1["Hidden Layer (128, ReLU)"]
    o         H2["Hidden Layer (64, ReLU)"]
    o         Output["Output (1: Vbmax)"]
    o     end
    o
    o     FT --> FTFeat
    o     VS --> VSFeat
    o     FTFeat --> Input88
    o     VSFeat --> Input88
    o     Input88 --> H1 --> H2 --> Output

```

A simple MLP (multi-layer perceptron) is appropriate here because:

- The dataset is small (likely ~50-60 usable cycles after intersection).
- Input is a fixed-length feature vector, not sequential data.
- A CNN or RNN would overfit with so few samples.

Given the small dataset size, we will use **Leave-One-Out Cross-Validation (LOOCV)** or **k-fold cross-validation** instead of a simple train/test split.

File Structure

All code will live in a single script:

`[tool_wear_prediction.py](tool_wear_prediction.py)` with clear sections. A `[requirements.txt](requirements.txt)` will also be created.

Implementation Steps

Step 1: Process tool wear.xls

- Read the Excel file with `pandas + xlrd`.
- Identify the Vbmax-related columns per cutting edge.
- Average wear width (Vbmax) across all edges for each cycle.
- Output: a DataFrame with columns `[Cycle, Vbmax_avg]`.

Step 2: Build file-to-cycle mapping

- List and sort all force/torque `.txt` files and vibration/sound `.csv` files chronologically by filename.
- Assign cycle numbers (1, 2, 3, ...) based on sorted order.
- Find the intersection: cycles that have both data types.
- Print the mapping table for manual verification.

Step 3: Extract features from force/torque files

- For each `.txt` file, read the tab-delimited data (Time, Fx, Fy, Fz, Mz).
- Compute the 11 statistical features for each of the 4 signal channels (Fx, Fy, Fz, Mz).
- Use `scipy.stats` for skewness/kurtosis, `numpy.fft` for frequency features.
- Result: 44-element feature vector per cycle.

Step 4: Extract features from vibration/sound files

- For each `.csv` file, read columns (3 accelerometers + 1 sound pressure).
- Compute the same 11 statistical features per channel.
- Result: 44-element feature vector per cycle.

Step 5: Merge and normalize

- Concatenate force/torque features and vibration/sound features into an 88-element vector per cycle.
- Merge with Vbmax_avg target by cycle number.
- Apply `StandardScaler` normalization to input features.

Step 6: Build and train the neural network

- Use PyTorch (already installed in the project's `.venv`).
- Architecture: `Linear(88 -> 128) -> ReLU -> Dropout(0.3) -> Linear(128 -> 64) -> ReLU -> Dropout(0.3) -> Linear(64 -> 1)`.
- Loss: MSE (regression task).
- Optimizer: Adam.
- Use k-fold cross-validation (k=5 or LOOCV) due to small dataset.
- Report RMSE and R-squared metrics.

Step 7: Visualization

- Plot predicted vs actual Vbmax.
- Plot Vbmax vs Cycle (actual and predicted).
- Plot training loss curves.
- GPT:

1. Time-domain features (your 8 features)

These describe the **shape and variability** of the signal in time.

You said:

- Mean
- Standard deviation
- RMS
- Max
- Min
- Peak-to-peak
- Skewness
- Kurtosis

Let's go one by one.

Mean

$$\text{Mean} = \frac{1}{N} \sum x_i \quad \text{Mean} = \frac{1}{N} \sum x_i$$

What it measures

The **average level** of the signal.

Physical meaning for machining

Depends on the signal:

- **Force** → average cutting force
- **Torque** → average spindle load
- **Sound/Vibration** → baseline energy level

Why it matters for tool wear

As the tool wears:

- friction increases
- cutting resistance increases
- force and torque increase

So:

Mean force and torque usually increase with wear.

✓ Very important feature

Always include.

Standard Deviation

$$\sigma = \sqrt{\frac{1}{N} \sum (x_i - \mu)^2} \quad \sigma = \sqrt{\frac{1}{N} \sum (x_i - \mu)^2}$$

What it measures

How much the signal fluctuates around the mean.

Physical meaning

Worn tools:

- cut less smoothly
- produce unstable forces
- create more vibration

So:

More wear → more variability

✓ Very important
Always include.

RMS (Root Mean Square)

$$\text{RMS} = \sqrt{\frac{1}{N} \sum x_i^2}$$

What it measures

Overall **energy level** of the signal.

Physical meaning

For vibration and sound:

- RMS strongly correlates with tool condition
- more wear → more vibration energy

Why it matters

This is one of the **most widely used wear indicators** in industry.

✓ Extremely important
Always include.

Max

$$\max(x)$$

What it measures

Largest spike in the signal.

Physical meaning

Worn tools:

- create occasional force spikes
- may chip material irregularly

These spikes appear as large maxima.

✓ Useful

But less stable than RMS.

Include it.

Min

$\min(x)$

What it measures

Lowest spike.

Physical meaning

Usually less meaningful than Max.

Still helps define signal range.

✓ Optional but fine to include.

Peak-to-Peak

$\text{Max} - \text{Min}$

What it measures

Full amplitude range.

Physical meaning

Worn tools:

- produce stronger oscillations
- larger vibration range

✓ Useful

Often more meaningful than Max alone.

Include.

Skewness

Asymmetry of signal\text{Asymmetry of signal}Asymmetry of signal

What it measures

Whether the signal distribution is:

- symmetric
- left-tailed
- right-tailed

Physical meaning

Wear introduces irregular events:

- impacts
- micro-chipping

These distort signal symmetry.

✓ Useful but secondary

Not critical at first.

Kurtosis

Peakedness of signal\text{Peakedness of signal}Peakedness of signal

What it measures

How many **extreme spikes** occur.

Physical meaning

Very important for:

- impact detection
- fault detection
- tool breakage

Worn tools:

- create sharp impact spikes
- increase kurtosis dramatically

✓ Very useful

Especially for vibration signals.

Summary of time-domain features

Feature	Importance	Why
Mean	★★★★	Load increases with wear
Std	★★★★	Instability increases
RMS	★★★★ ★	Strong wear indicator
Max	★★★★	Detect spikes
Min	★★	Minor support feature
Peak-to-peak	★★★★ ★	Vibration range
Skewness	★★★★	Shape change
Kurtosis	★★★★ ★	Detect impacts

2. Frequency-domain features

Now we move into **FFT-based features**.

These describe **how energy is distributed across frequencies**.

Very important for vibration and sound.

You listed:

- Dominant frequency
- Spectral centroid
- Total spectral energy

Let's explain.

Dominant Frequency

From FFT:

f_{max}

Frequency with largest amplitude.

Physical meaning

Represents:

- spindle rotation frequency
- vibration resonance
- chatter frequency

Why it matters

As tools wear:

- chatter becomes more likely
- new dominant frequencies appear

✓ Very useful
Especially for vibration.

Spectral Centroid

$$\text{Centroid} = \frac{\sum f_i A_i}{\sum A_i}$$

What it measures

The **center of mass** of the spectrum.

Physical meaning

Tells whether signal energy is:

- low-frequency heavy
- high-frequency heavy

Wear tends to shift energy toward:

- higher frequencies

Because:

- friction increases
- impacts increase

✓ Very useful
Often used in audio analysis too.

Total Spectral Energy

$$\sum |FFT(x)|^2$$

What it measures

Total energy in frequency domain.

Physical meaning

More wear:

→ more vibration energy

✓ Very important

Especially for vibration and sound.

-